AGROVET AI

Multi-Domain Computer Vision Platform for Agricultural Pathology & Veterinary Diagnostic Imaging

Document Title:	Technical Specifications & Project Report
Software Version:	1.0.0 (Production Release)
Release Date:	August 06, 2026
Authors:	AI Architecture Group & Lead Developer
System Core:	React (Vite) / Python FastAPI / PyTorch CNN / SQLite / Docker

Executive Summary

AgroVet AI is an integrated, full-stack artificial intelligence application engineered to support two distinct classification domains using deep learning: Agricultural Foliar Leaf Pathology (detecting 22 crop disease combinations) and Veterinary Orthopedic Radiography (detecting 14 animal trauma anomalies).

By hosting both domains within a single containerized system, AgroVet AI provides immediate field diagnostic results with built-in Explainable AI (XAI) using Gradient-Weighted Class Activation Mapping (Grad-CAM). The system requires no user authentication for its core operational functions, resolving adoption issues for rural agricultural users and veterinary clinicians in high-stress field conditions. Scans are logged locally in an SQLite database, compiled into an administrative telemetry dashboard, and can be rendered on-demand as professional, publication-quality diagnostic PDFs.

Table of Contents

1. Introduction & Background

1.1 Operational Domain Context

In modern agrarian and veterinary medicine, rapid diagnostics translate directly into damage control. Agricultural extension workers and farmers often wait days or weeks for laboratory foliar tissue analysis, allowing fungal diseases like Early Blight or Late Blight to ruin entire fields. In contrast, remote veterinary shelters or rural clinics face immediate clinical situations where injured animals (dogs, cats, cows, horses) require initial trauma evaluations. Immediate radiographic screenings for bone fractures or joint abnormalities allow staff to splint limbs and prepare treatment plans before a specialist veterinarian arrives.

1.2 The AgroVet AI Unified Diagnostic Framework

AgroVet AI acts as a single, accessible gateway for local diagnostic imaging, designed to solve several operational bottlenecks simultaneously:

- **No-Login Mandate:** Deployed as a friction-free utility, field workers do not need to register, remember credentials, or connect to OAuth services under unstable cellular data coverage. The core diagnostic page runs immediately in any browser.
- **Double-Domain Processing:** Combines plant disease pathology and veterinary bone analysis under one API routing engine, maximizing container resource utilization.
- **Explainable Visual Evidence:** Rather than returning a single classification class and a confidence score, the system generates attention maps showing where the model located leaf lesions or bone fractures, helping prevent 'black-box' trust failures.
- **Documentable Data:** Generates professional PDFs that can be printed, archived, or shared via email with diagnostic experts for validation.

2. System Architecture & Data Flow

2.1 High-Level Component Layout

The system is organized into a React Single Page Application (SPA) frontend built on Vite, communicating via asynchronous JSON and binary payloads with a Python FastAPI server. The database is SQLite, stored inside the backend container to ensure zero external dependencies during standard local deployment.

Component	Tech Stack Details	Operational Function
Frontend Client	React 18, Vite, Custom CSS (NVIDIA Dark Theme), Lucide Icons	Handles client-side routing, drag-and-drop uploads, interactive sample selection, SVG metric rendering, and modal details collection.
API Gateway Server	Python FastAPI, Uvicorn Server, CORS Middleware	Manages routing, async execution pools, file ingestion, static content directories, and acts as the orchestrator for inference and PDF compilation.
Deep Learning Engine	PyTorch 2.0+, Torchvision, PIL, NumPy	Loads trained SimpleCNN weights, preprocesses incoming tensors, computes class logits, and processes backward gradient hooks.
Explainability (XAI)	OpenCV (cv2), Grad-CAM mathematical overlays	Extracts activation feature maps and gradients, blends a jet colormap with the original image, and exports JPEGs.
Persistence & Logging	SQLite3 database, native Python SQLite driver	Stores analysis records, files, prediction confidences, dates, and updates user details for historical report tracking.
PDF Compilation	ReportLab Flowables API (SimpleDocTemplate)	Compiles database records into double-channel imagery layouts, structured text flowables, and exports PDF files.

2.2 Dynamic Request-Response Lifecycle

A typical client-server loop begins with a REST request to either `/api/diagnose/plant` or `/api/diagnose/animal`. The file is saved locally to generating a UUID name. The inference engine performs classification, passes the outputs to the Grad-CAM module to build the attention map, records the statistics in the SQLite database, and returns the metadata to the React app in a single, high-performance request-response cycle.

3. Deep Learning Engine & Model Training

3.1 Neural Network Design: SimpleCNN

Both diagnostic domains utilize a custom 3-layer Convolutional Neural Network (CNN) class (`SimpleCNN`) that inherits from `torch.nn.Module`. This architecture balances classification performance with rapid, lightweight training execution, suitable for local setups without dedicated GPUs.

- **Conv2d Layers:** The network registers three sequential 2D convolutional layers. Conv1 converts the 3-channel input to 16 feature maps. Conv2 expands this to 32 channels, and Conv3 finishes with 64 channels. Each layer uses a 3x3 kernel with padding of 1 to preserve border spatial dimensions.
- **Max Pooling:** 2x2 Max Pooling layers downsample the activation grids by half after each convolution block, reducing a 224x224 input to 112x112, 56x56, and finally 28x28 pixels.
- **Fully Connected Layers:** The final 64x28x28 tensor is flattened into a 1D vector of 50,176 elements. A Linear layer maps this to a hidden layer of 128 elements, and the final classification layer maps to the category logits.

3.2 Synthetic Data Training Pipeline

To allow instant local operation without requesting massive datasets, the backend includes `train_models.py` which auto-generates simulated datasets to train the PyTorch networks:

1. **Leaf Anomaly Simulation:** Synthesizes RGB image arrays representing green circular leaves. Diseased leaves are generated with random clusters of brown and yellow pixels to represent foliar lesions.
2. **Bone Trauma Simulation:** Synthesizes grayscale limb shapes against black backgrounds. Bone fractures are simulated as jagged lines of high contrast.
3. **Model Export:** The networks are trained using the Cross-Entropy loss function and Adam optimizer for 2 epochs, saving the final parameters to `plant_model.pth` and `xray_model.pth`.

4. Explainable AI: Grad-CAM Implementation

4.1 Principles & Core Mathematics

Standard convolutional networks are often critiqued as 'black boxes' because their hidden decisions lack spatial visibility. AgroVet AI resolves this by overlaying Gradient-Weighted Class Activation Maps (Grad-CAM).

Grad-CAM uses the gradients flowing back from the predicted class logit (y^c) to the last convolutional layer's activation map (A^k) to compute an attention score for each filter. This highlights the spatial structures that drive the model's prediction.

First, we calculate the channel weight alpha ($■lpha_k^c$) by averaging the gradients over the height v and width u of the feature maps:

$$\alpha_k^c = (1 / Z) \Sigma_i \Sigma_j (\partial y^c / \partial A_{i,j}^k)$$

Next, we compute a weighted linear combination of all forward activations A^k and apply the Rectified Linear Unit (ReLU) to isolate features that positively contribute to the target class:

$$L^c_{Grad\text{-}CAM} = ReLU(\Sigma_k \alpha_k^c A^k)$$

4.2 Interception Hooks and OpenCV Image Overlay

During model forward propagation, the network registers a gradient hook on the final conv layer using `x.register_hook(save_gradient)`. Once the backward gradient pass runs, these gradients are captured. The raw 2D activation matrix is normalized, resized using bilinear interpolation to match the dimensions of the original image, converted to a pseudo-color JET colormap, and blended with the original BGR image:

Overlay Blend Formula: $$Image_{Overlay} = 0.6 \cdot Image_{Original} + 0.4 \cdot Heatmap_{Colored}$$

5. Database & Admin Analytics Dashboard

5.1 Database Schema & Data Access Layer

To minimize deployment dependencies, data persistence is handled by a single SQLite database file (`backend/agrovet.db`). The database schema is designed for rapid diagnostic logging and telemetry parsing, tracking the raw file paths and generated Grad-CAM heatmap overlays.

Field Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique sequence identifier for the analysis entry.
user_name	TEXT	NULL	Demographic name collected before PDF download.
user_mobile	TEXT	NULL	Optional contact phone number.
user_email	TEXT	NULL	Optional electronic mail contact.
type	TEXT	NOT NULL	Classification domain: 'plant' or 'animal'.
target_type	TEXT	NOT NULL	Subject species/crop type (e.g., Dog, Tomato).
prediction	TEXT	NOT NULL	Classified pathology name or medical finding.
confidence	REAL	NOT NULL	Softmax probability value (0.0 to 1.0).
image_path	TEXT	NOT NULL	Local server path to original image.
heatmap_path	TEXT	NOT NULL	Local server path to Grad-CAM output.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Auto-generated submission time.

5.2 Telemetry Dashboard Interface

The server aggregates database records via `get_dashboard_stats()` to build the admin dashboard. To ensure a zero-dependency setup, the React client renders these trends using custom SVG vector graphics. The system draws line grids, dates, and glowing area polygons representing the monthly diagnostic load directly in the browser.

6. PDF Report Generation System

6.1 Programmatic Document Compilation

AgroVet AI includes a high-fidelity PDF generation engine in `report_generator.py` powered by the ReportLab flowables pipeline. When a user requests a report, the server compiles the metadata, database fields, and local images into a structured PDF document.

6.2 Layout Design & Flowables Flow

- **Branding Banner:** A structured table draws a charcoal background banner containing the 'AGROVET AI' title and domain details. A solid border matches the domain: Neon Green for agricultural reports, and Neon Blue for animal reports.
- **Demographic Metadata Grid:** A structured table formats the user name, email, contact phone, analysis date, and target subject parameters in a clean 2-column grid.
- **Double-Channel Visuals:** Renders the original uploaded image next to the Grad-CAM heatmap overlay. Images are constrained to $240 ext{ pt} imes 200 ext{ pt}$ boundaries to fit the margins without page overflow.
- **Structured Diagnoses:** Integrates symptom breakdowns, environmental causes, and action/treatment plans matching the predicted category.
- **Legal Disclaimer:** Every generated PDF includes a prominent, italicized footer warning that the AI-generated report is intended for preliminary screening and should be confirmed by a specialist veterinarian or plant pathologist.

6.3 Streamed Delivery Pipeline

The PDF generation process runs synchronously within FastAPI. Once the document build is complete, the file path is wrapped in a FastAPI `FileResponse` object. This streams the raw PDF binary back to the client browser, prompting a native download dialog named after the user (e.g., `Dilip_Y_AgroVet_Report.pdf`).

7. User Interface Design & Styling

7.1 Design Philosophy: NVIDIA-Style Glow

The AgroVet AI portal is designed with a premium, dark-mode-first aesthetic inspired by modern compute platforms like NVIDIA. Styling is defined in `src/index.css` using CSS custom variables to manage color palettes and interactive glowing borders.

CSS Selector / Token	Value / Colors	Visual Application
--bg-dark	#0a0f1d (Deep slate black)	Primary background color for the application viewport.
--card-bg	rgba(17, 24, 39, 0.7) (Translucent gray)	Background color for glassmorphic cards and containers.
--glow-green	#00e676 (Neon green)	Primary theme accent for plant diagnosis pages and success states.
--glow-blue	#00b0ff (Neon blue)	Primary theme accent for animal X-ray pages and metric highlights.
--text-light	#f3f4f6 (Off-white)	Primary readable body text color.
backdrop-filter	blur(12px)	Renders a glassmorphic blurred background behind transparent cards.

7.2 Gated Report Acquisition Flow

To encourage data logging without creating friction, the platform does not require registration to perform scans. However, to download a PDF report, the interface prompts the user with a glassmorphic modal requesting their Name (Mandatory), Mobile Number (Optional), and Email (Optional). Once submitted, the backend logs these details and streams the generated report.

8. Deployment Framework & Roadmap

8.1 Containerized Docker Deployment

To ensure AgroVet AI runs reliably across diverse desktop, tablet, and mobile browsers, the application is packaged in a multi-stage Docker container. Stage 1 compiles the React frontend assets, and Stage 2 sets up the Python backend server. The compiled React files are copied into the backend's static directories, allowing FastAPI to serve both the API and the user interface from a single port (8000).

8.2 Optimization & Performance

- **Model Evaluation Mode:** The PyTorch models are initialized and locked in evaluation mode (`model.eval()`) on startup, disabling gradient caching for forward passes. This keeps memory usage low and response times fast during concurrent user requests.
- **Static Asset Caching:** Image files and generated Grad-CAM heatmaps are saved under the mounted static `/data` directory. These assets are cached by the browser to reduce redundant loads during UI navigation.

8.3 Future Roadmap & Research Directions

1. **Advanced Backbone Models:** Upgrade the custom SimpleCNN classification backend to pre-trained transfer learning architectures like ResNet-50 or MobileNet-V3. This will improve prediction accuracy on complex, real-world leaf lesions and veterinary X-rays.
2. **Web Assembly (Wasm) Portability:** Export trained PyTorch weights to the ONNX format, allowing the models to run directly in the client browser. This enables offline field diagnostics under poor cellular coverage.
3. **Multi-Spectral Image Analysis:** Extend the agricultural model to parse multi-spectral satellite or drone imagery, enabling early crop disease detection before leaf damage becomes visible to RGB cameras.